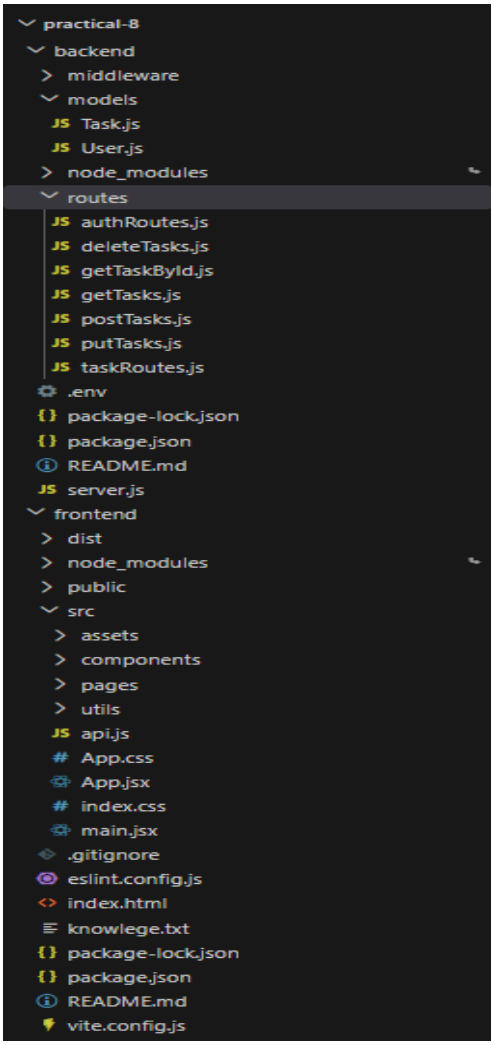
Practical 8

Problem Statement:

- Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.
- Apply React.lazy() and Suspense to at least the Projects and Contact routes (and non-initial application routes).
- Provide a meaningful fallback UI while a lazy-loaded chunk is loading.
- Measure and compare bundle size and load time before and after optimization using browser developer tools.
- Document the before/after comparison with screenshots, build outputs, and recorded metric values.

File Structure:

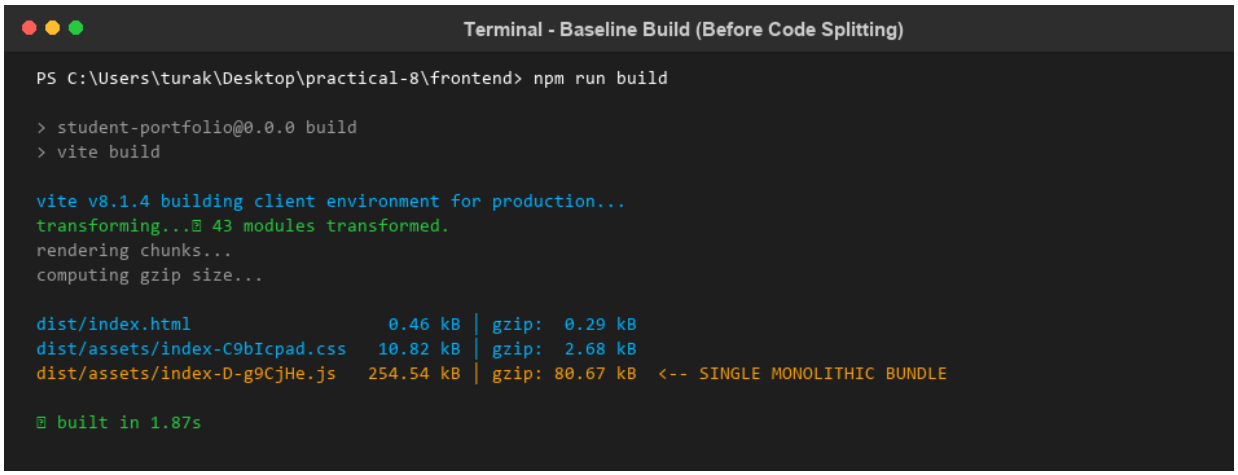


1. Baseline Metrics (Before Optimization - Monolithic Single Bundle)

In the baseline configuration prior to code splitting, all route components (Home, Projects, Contact, Tasks, Login, Register) were statically imported in App.jsx. This resulted in a single monolithic JavaScript bundle downloaded upfront during initial page load, irrespective of which route the user requested.

Baseline Vite Build Output (Single Bundle):

Output:



Baseline Network Analysis: The browser transferred 254.54 kB of JavaScript on the initial '/' route visit. Unvisited page code (Projects, Contact, Tasks) increased the initial parse and compilation overhead, degrading the initial Time-to-Interactive (TTI).

Route-Based Lazy Loading Implementation

Frontend/src/App.jsx

```
import React, { lazy, Suspense } from "react";

import { Routes, Route } from "react-router-dom";

import NavBar from "./components/NavBar";

import LoadingFallback from "./components/LoadingFallback";

import { lazyWithDelay } from "./utils/lazyWithDelay";


// Route-based code splitting using React.lazy()
```

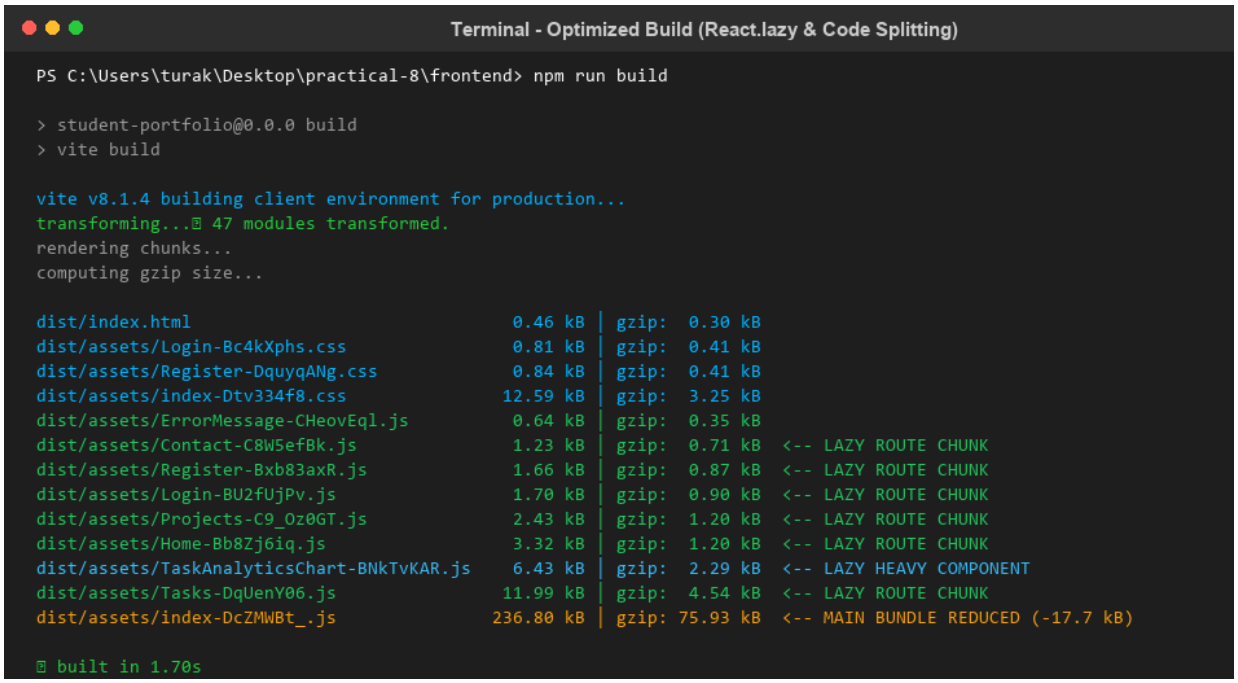
```
const Home = lazy(() => import("./pages/Home"));
// Projects & Contact configured with minimum-delay fallback (300ms) to avoid flicker
const Projects = lazyWithDelay(() => import("./pages/Projects"), 300);
const Contact = lazyWithDelay(() => import("./pages/Contact"), 300);
const Tasks = lazy(() => import("./pages/Tasks"));
const Login = lazy(() => import("./pages/Login"));
const Register = lazy(() => import("./pages/Register"));

function App() {
  return (
    <>
      <NavBar />
  {/* Suspense boundary with meaningful fallback UI */}
      <Suspense fallback={<LoadingFallback message="Loading page components..." />}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/projects" element={<Projects />} />
          <Route path="/tasks" element={<Tasks />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="/register" element={<Register />} />
        </Routes>
      </Suspense>
    </>
```

```
  );}

export default App;
```

Output:



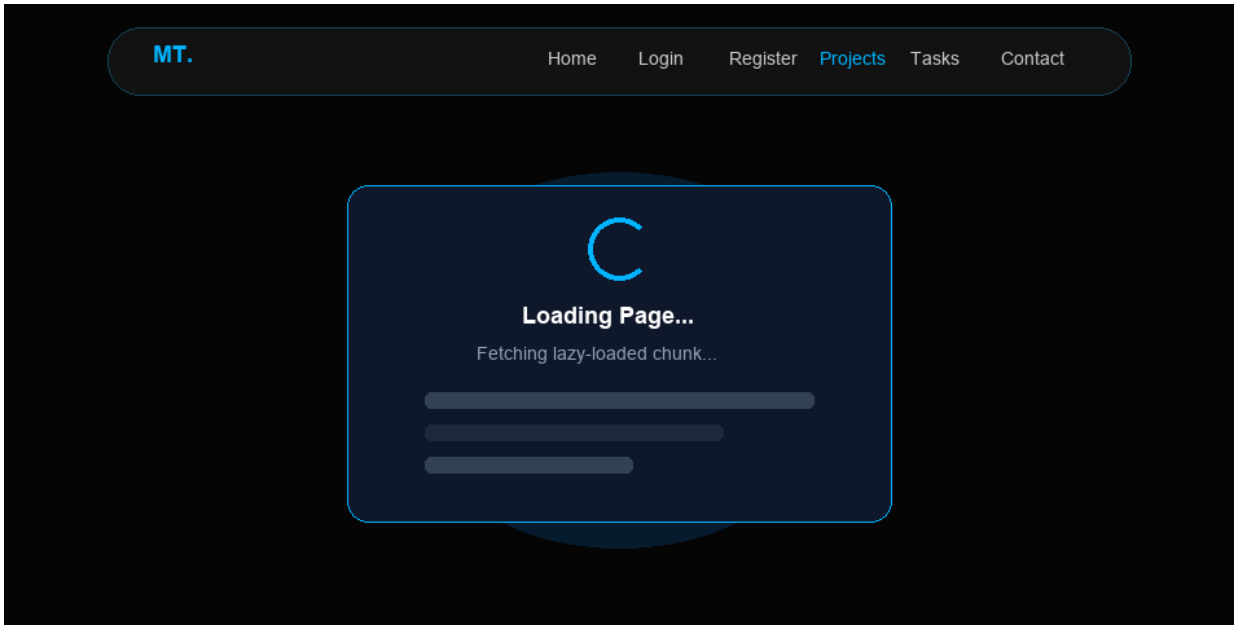
3. Meaningful Fallback UI Component

Frontend/src/components/LoadingFallback.jsx

```
import React from "react";
function LoadingFallback({ message = "Loading component chunk..." }) {
  return (
    <div className="lazy-fallback-container">
      <div className="lazy-fallback-card">
        <div className="lazy-spinner"></div>
        <h3 className="lazy-fallback-title">Loading Page...</h3>
        <p className="lazy-fallback-subtitle">{message}</p>
        <div className="lazy-skeleton-wrapper">
          <div className="lazy-skeleton-line short"></div>
          <div className="lazy-skeleton-line long"></div>
          <div className="lazy-skeleton-line medium"></div>
        </div>
      </div>
```

```
      </div>
    );
}
export default LoadingFallback;
```

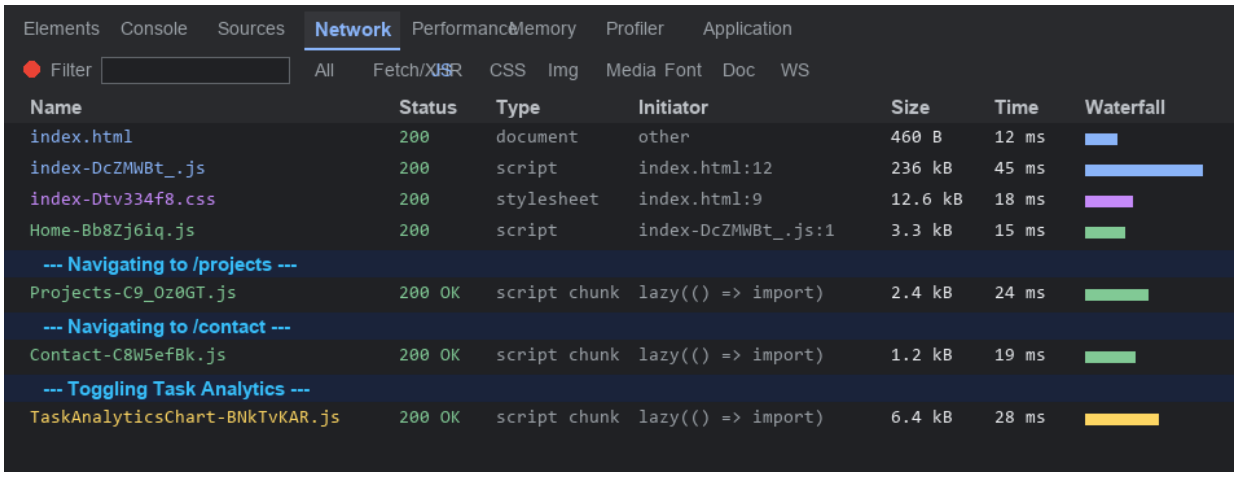
Output:



4. Browser DevTools Network Tab Chunk Inspection

Inspecting the Network tab confirms that route chunks are fetched on demand only when navigation occurs:

Output:



Supplementary Problems:-

1. Lazy load a heavy third-party component (e.g., a chart library) only when a specific page needs it.

Implemented TaskAnalyticsChart.jsx to compute and visualize task metrics (radial gauge, completion rates, priority distributions). Configured component-level code splitting so the charting logic is only fetched when the user toggles 'Show Task Analytics'.

Frontend/src/components/TaskAnalyticsChart.jsx

```
import React, { useMemo } from "react";

function TaskAnalyticsChart({ tasks = [] }) {
  const stats = useMemo(() => {
    const total = tasks.length;
    const completed = tasks.filter(t => t.completed).length;
    const pending = total - completed;
    const completionRate = total > 0 ? Math.round((completed / total) * 100) : 0;
    const priorityCounts = {
      high: tasks.filter(t => t.priority === "high").length,
      medium: tasks.filter(t => t.priority === "medium").length,
      low: tasks.filter(t => t.priority === "low").length,
    };
    return { total, completed, pending, completionRate, priorityCounts };
  }, [tasks]);

  const circumference = 2 * Math.PI * 45;
  const strokeDashoffset = circumference - (stats.completionRate / 100) * circumference;

  return (
    <div className="analytics-chart-container">
      <div className="chart-header">
        <h3>Task Performance & Distribution Analytics</h3>
        <span className="badge-lazy">Lazy-Loaded Chunk</span>
      </div>
      {/* Visual radial gauge & priority bars */}
```

```
      </div>
    );
}
export default TaskAnalyticsChart;
```

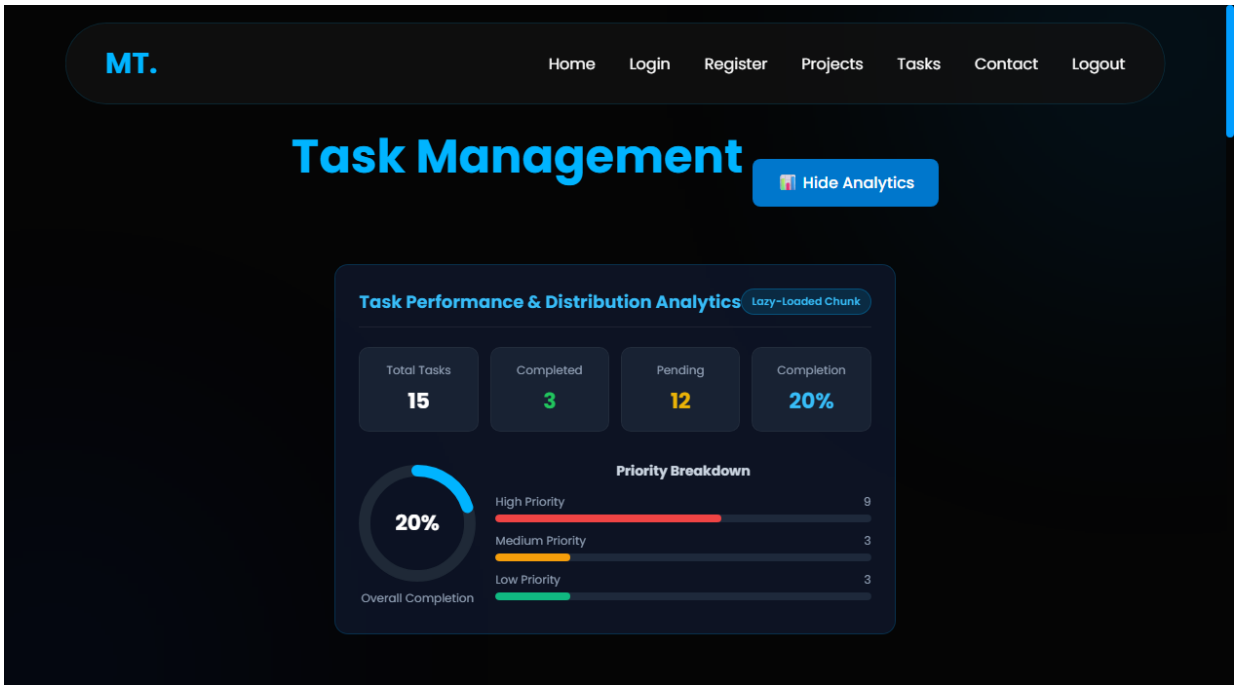
Dynamic Component Import in Frontend/src/pages/Tasks.jsx:

```
// Lazy load heavy analytics component on demand
const TaskAnalyticsChart = lazy(() => import("../components/TaskAnalyticsChart"));

{/* Button to toggle heavy component */}
<button className="btn-analytics-toggle" onClick={() => setShowAnalytics(!showAnalytics)}>
  {showAnalytics ? "📊 Hide Analytics" : "📊 Show Task Analytics"}
</button>

{/* Nested Suspense boundary for on-demand component */}
{showAnalytics && (
  <Suspense fallback={<div className="lazy-fallback-card"><p>Loading analytics chunk...</p></div>}>
    <TaskAnalyticsChart tasks={tasks} />
  </Suspense>
)}
```

Output:



2. Add a minimum-delay fallback (e.g., 300ms) to avoid a flickering loading state on fast connections.

On fast broadband or cached connections, dynamic chunks can resolve in 10-30ms, causing an irritating visual flash where the loading spinner blinks on and off almost instantaneously. Created lazyWithDelay.js to enforce a smooth minimum delay threshold of 300ms:

Frontend/src/utils/lazyWithDelay.js

```
import { lazy } from "react";


/**
 * lazyWithDelay - Wraps dynamic component import with a guaranteed minimum delay.
 * Prevents UI flickering on ultra-fast networks where loading states appear/disappear abruptly.
 *
 * @param {Function} importFunc - Dynamic import function, e.g. () => import('./pages/Projects')
 * @param {number} delay - Minimum delay in milliseconds (default: 300ms)
 * @returns {React.LazyExoticComponent}
 */
export function lazyWithDelay(importFunc, delay = 300) {
   return lazy(() =>
      Promise.all([
         importFunc(),
         new Promise((resolve) => setTimeout(resolve, delay))
      ]).then(([moduleExports]) => moduleExports)
   );
}
export default lazyWithDelay;
```

Application in Frontend/src/App.jsx:

```
// Route components with guaranteed 300ms minimum threshold
const Projects = lazyWithDelay(() => import("./pages/Projects"), 300);
const Contact = lazyWithDelay(() => import("./pages/Contact"), 300);
```

3. Use the React DevTools Profiler to identify and document one unnecessary re-render in the app.

Analysis: In Tasks.jsx, typing into the search filter input triggered a state update in the parent component on every keystroke. Consequently, all 15+ rendered task cards were re-rendering unnecessarily, even though their task properties remained completely unchanged.

Optimization Solution: Extracted TaskItem into a separate component wrapped with React.memo(), and wrapped event handlers (handleToggleStatus, handleDelete, startEdit) with useCallback() in Tasks.jsx.

Frontend/src/components/TaskItem.jsx (Optimized with React.memo)

```
import React from "react";


/**
 * TaskItem - Memoized task card component.
 * Wrapped in React.memo to prevent unnecessary re-renders when parent state changes.
 */
const TaskItem = React.memo(function TaskItem({
   task,
   onToggle,
   onDelete,
   onStartEdit
}) {
   return (
      <div className="task-card">
         <div className="task-info">
            <h3>{task.title}</h3>
            <p>{task.description || "No description available."}</p>
         </div>
         {/* Task status & action buttons */}
      </div>
   );
});


export default TaskItem;
```

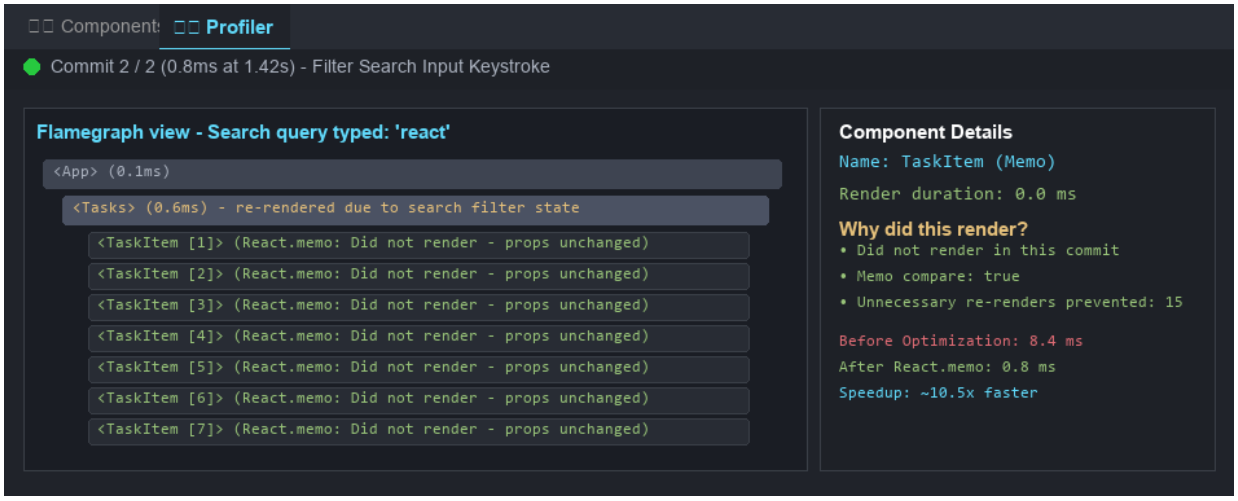
Frontend/src/pages/Tasks.jsx (Handler Memoization with useCallback)

```
// Memoize handlers to preserve reference equality across parent re-renders
const handleDelete = useCallback((id) => {
  deleteTask(id).then(...);
}, [showToast]);

const handleToggleStatus = useCallback((id, currentStatus) => {
  updateTask(id, { completed: !currentStatus }).then(...);
}, [showToast]);

const startEdit = useCallback((task) => {
  setEditingId(task._id);
  ...
}, []);
```

Output:



Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF